

Лабораторная работа №5.

«Компонентный подход и строгая типизация в React»

Тема: Разработка библиотеки базовых UI-компонентов (UI Kit) для системы мониторинга.

Стек: React, TypeScript, CSS Modules.

Задание

Реализовать 4 базовых компонента.

1. Компонент `Button`

- **Функционал:** Кнопка с поддержкой различных состояний.
- **Props:**
 - `variant: 'primary' | 'secondary' | 'danger'.`
 - `size: 'small' | 'medium' | 'large'.`
 - `isLoading: boolean` (если `true`, показывать текст "Загрузка..." и блокировать кнопку).
 - Стандартные атрибуты `HTMLButtonElement`.
- **Особое требование:** Используйте `React.ButtonHTMLAttributes<HTMLButtonElement>` для наследования стандартных свойств.

2. Компонент `Input`

- **Функционал:** Поле ввода с подписью (Label) и выводом ошибки.
- **Props:**
 - `label: string.`
 - `error: string` (опционально).
 - `isFullWidth: boolean.`
- **Особое требование:** Компонент должен быть «контролируемым».

3. Компонент `Badge` (Индикатор статуса)

- **Функционал:** Маленький индикатор для отображения ролей или статусов (например, "admin", "online").
- **Props:**
 - `color: 'green' | 'red' | 'orange' | 'blue'.`
 - `text: string.`

4. Компонент `LayoutCard`

- **Функционал:** Обертка для контента с заголовком и футером.
- **Props:**
 - `title: ReactNode.`
 - `children: ReactNode.`
 - `footer: ReactNode` (опционально).

Требования к реализации

Типизация

Запрещено использовать `any`. Если вы не знаете тип, обратитесь к документации React или используйте встроенные типы (например, `React.ChangeEvent<HTMLInputElement>`).

Структура папок

Каждый компонент должен находиться в своей папке внутри `src/components/ui/`:

```
Plaintext
/Button
├── Button.tsx
├── Button.module.css
└── index.ts (для чистого импорта)
```

Стилизация

Используйте библиотеку `classnames` для удобного объединения классов в зависимости от пропсов.

Теория

UI Kit: Фундамент масштабируемых интерфейсов

В современной разработке создание приложения «с чистого листа» для каждой страницы — это путь к техническому долгу и визуальному хаосу. Решением этой проблемы является **UI Kit**.

Что такое UI Kit?

UI Kit (User Interface Kit) — это библиотека стандартизированных визуальных элементов, стилей и компонентов, из которых собирается интерфейс. В контексте React-разработки — это набор «глупых» (**stateless**) компонентов, которые отвечают исключительно за отображение и не привязаны к бизнес-логике.

Анатомия UI Kit: От атомов к организмам

Согласно методологии **Atomic Design**, UI Kit строится иерархично:

1. **Design Tokens (Основа)** Это «переменные» дизайна: цвета (hex-коды), размеры шрифтов, отступы (`margin/padding`), радиусы скругления и параметры теней. В коде они часто реализуются через CSS-переменные или объекты в TypeScript.
2. **Atoms (Атомы)** Простейшие неделимые компоненты: кнопки (`Button`), поля ввода (`Input`), иконки, чекбоксы.
3. **Molecules (Молекулы)** Группы атомов, работающие вместе. Например, поисковая строка, состоящая из `Input` и `Button`.
4. **Organisms (Организмы)** Сложные блоки интерфейса: шапка сайта (`Header`), карточка товара с кнопками и рейтингом, или навигационная панель.

Технические требования к качественному UI Kit

При написании UI Kit на React и TypeScript мы придерживаемся четырех столпов:

1. Строгая типизация (Type Safety)

Компоненты должны иметь четко описанные интерфейсы. Использование `any` запрещено, так как оно разрушает «контракт» между компонентами.

- *Пример:* Использование `React.ButtonHTMLAttributes<HTMLButtonElement>` позволяет кнопке наследовать все стандартные свойства HTML-элемента автоматически.

2. Инкапсуляция стилей

Стили одного компонента не должны влиять на другие. Мы используем **CSS Modules**, которые генерируют уникальные имена классов, предотвращая каскадные конфликты.

3. Предсказуемость (Pure Components)

Компоненты UI Kit должны быть «чистыми»: при одних и тех же пропсах они всегда выдают один и тот же результат. Они не должны делать запросы в API или обращаться к глобальному состоянию (например, Recoil) напрямую.

4. Документированность

Разработчик должен понимать, как использовать компонент, не заглядывая в его исходный код. Популярный инструмент для этого — **Storybook**, позволяющий просматривать компоненты в изолированной среде.

Почему это важно для вашего проекта?

1. **Единообразие (Consistency):** Пользователь не запутается, так как все кнопки «Удалить» в приложении будут выглядеть и вести себя одинаково.
2. **Скорость (Velocity):** Сборка новой страницы превращается в компоновку готовых блоков, а не в написание CSS с нуля.
3. **Легкость рефакторинга:** Нужно изменить скругление у всех полей ввода? Вы меняете одну переменную в UI Kit, и изменения применяются ко всему приложению.

CSS Modules

Одной из самых больших проблем классического CSS является его **глобальность**. Если вы определили класс `.button` в одном файле, он может случайно переопределить стиль кнопки в совершенно другой части приложения.

CSS Modules решают эту проблему на уровне сборки, превращая CSS в систему локальных пространств имен.

1. Что это такое?

CSS Modules — это не новый язык, это технология, которая при сборке (например, через Vite или Webpack) автоматически изменяет имена ваших классов на уникальные хеши.

Вместо того чтобы писать глобальный селектор, вы импортируете объект со стилями прямо в JS-файл.

2. Как это работает на практике

Шаг 1: Создание файла

Файл обязательно должен иметь расширение `.module.css`.

CSS

```
/* Button.module.css */
.button {
  background-color: #007bff;
  padding: 10px 20px;
  border-radius: 4px;
}

.loading {
  opacity: 0.6;
  cursor: not-allowed;
}
```

Шаг 2: Импорт в React

Вы импортируете стили как обычный JavaScript-объект.

TypeScript

```
import s from './Button.module.css';

export const Button = ({ isLoading, children }) => {
  // s — это объект, где ключи — ваши названия классов,
  // а значения — сгенерированные уникальные строки.
  return (
    <button className={` ${s.button} ${isLoading ? s.loading : ''}`}>
      {children}
    </button>
  );
};
```

Что увидит браузер в итоге? Вместо `class="button"`, в инспекторе вы увидите нечто вроде `class="_button_1a2b3_5"`. Это гарантирует, что стили этого компонента никогда не пересекутся с другими.

3. Ключевые преимущества

Полная изоляция (Encapsulation)

Вы можете называть классы просто: `.container`, `.wrapper`, `.item`. Вам больше не нужны длинные имена по методологии BEM (`.header__navigation-list--item`), так как хеш обеспечит уникальность.

Безопасный рефакторинг

Если вы удалите компонент и его `.module.css`, вы можете быть на 100% уверены, что не сломали стили в другом месте.

Работа с TypeScript

Вы можете типизировать CSS-модули, чтобы IDE подсказывала вам названия существующих классов и выдавала ошибку, если вы опечатались в имени класса.

4. Продвинутые приемы

Глобальные стили внутри модуля

Иногда нужно стилизовать стороннюю библиотеку или тег `body`. Для этого используется псевдоселектор `:global`:

CSS

```
/* Этот класс не будет хеширован */
:global(.active) {
  color: red;
}
```

Композиция классов

Вы можете наследовать стили внутри CSS-файла, уменьшая дублирование кода:

CSS

```
.base {
  display: inline-flex;
}

.primary {
  composes: base;
  background: blue;
}
```

5. Работа с динамическими классами

Для удобного объединения классов (особенно когда их много и они зависят от условий) в React-сообществе принято использовать библиотеку `clsx` или `classnames`.

TypeScript

```
import cn from 'classnames';
import s from './Button.module.css';

const btnClass = cn(s.button, {
```

```
[s.primary]: variant === 'primary',  
[s.danger]: variant === 'danger',  
[s.disabled]: isDisabled  
});
```

Контрольные вопросы:

1. **Что такое UI Kit** и какую основную проблему в разработке интерфейсов он решает?
2. Объясните концепцию **Atomic Design**. Чем «атом» отличается от «молекулы» в контексте React-компонентов?
3. Что такое **Design Tokens**? Приведите примеры того, какие параметры дизайна могут быть токенами.
4. В чем заключается принцип «глупых» (**stateless**) компонентов? Почему компоненты UI Kit не должны содержать бизнес-логику?
5. Зачем при создании компонента `Button` расширять интерфейс от `React.ButtonHTMLAttributes<HTMLButtonElement>`?
6. Какой тип данных в TypeScript лучше всего подходит для пропса `children` и почему?
7. Какую главную проблему классического CSS решают **CSS Modules**?
8. Каким образом CSS Modules гарантируют уникальность имен классов в разных компонентах?
9. Для чего используется библиотека `classnames` (или `clsx`) при разработке компонентов?
10. Как внутри CSS-модуля объявить глобальный стиль, который не должен подвергаться хешированию?
11. Что означает требование о том, что компонент `Input` должен быть «контролируемым»?